

Vedashri Travels

Deploy on Render, Railway and a VPS

Detailed note for putting this Node.js app online so the three links stay up and bookings do not reset. Written for the current code: Express on port from process.env.PORT, start command npm start, PostgreSQL when DATABASE_URL is set, otherwise data/store.json.

1. What you are deploying

One Node.js server, three public pages:

- Customer desktop: /customer.html (same as /)
- Owner dashboard: /owner.html (PIN 2145 until you change it)
- Customer mobile app: /app.html

This is not a static site. Do not use Vercel, Netlify or GitHub Pages. You need a host that keeps a Node process running.

After deploy, share

```
https://YOUR-HOST/customer.html
https://YOUR-HOST/owner.html
https://YOUR-HOST/app.html
```

2. How data is stored

- If DATABASE_URL is set: PostgreSQL table app_state (company, cars, bookings, staff).
- If DATABASE_URL is empty: file data/store.json (or \$DATA_DIR/store.json).
- Uploaded logos, QR and car photos go in DATA_DIR/uploads.

On Render Free without Postgres, store.json is wiped on every restart or redeploy. That is why bookings and fleet reset. Link a PostgreSQL database. Then you do not need a paid disk.

3. Push the project to GitHub

1. Create a GitHub account if needed. New repository: vedashri-travels. Do not add a README on GitHub if this folder already has one.
2. On your computer, in the project folder, run the commands below. Replace YOUR_USER.
3. GitHub will ask for a personal access token as the password, not your GitHub login password.
4. Confirm the repo shows public/customer.html, public/owner.html, public/app.html, server.js and package.json.

```
git add .
git commit -m "Vedashri Travels ready to deploy"
git branch -M main
git remote add origin https://github.com/YOUR_USER/vedashri-travels.git
git push -u origin main
```

Later updates (same repo, same Render/Railway service):

```
git add .
git commit -m "Describe the change"
git push
```

4. Deploy on Render (recommended)

Use the existing Web Service if you already created one. Do not click New Web Service unless you want a second copy.

4.1 Create or open the Web Service

1. Sign in at render.com with GitHub.
2. If you have no service yet: New + -> Web Service -> connect vedashri-travels.
3. If you already have a service: open that service. Skip creating another one.

Web Service settings

Runtime / Language: Node

Branch: main

Build command: npm install

Start command: npm start

Do not set PORT. Render sets it. server.js already reads process.env.PORT.

Click Create Web Service (or Save). Wait until logs show: Vedashri Travels running on...

4.2 Add PostgreSQL so data does not reset

1. In the Render dashboard click New + -> PostgreSQL.
2. Name: vedashri-db. Region: same region as the web service. Plan: Free is enough to start.
3. Create the database. Wait until status is Available.
4. Open the database -> Connect. Copy Internal Database URL (for services on Render).
5. Open your Web Service -> Environment.
6. Add DATABASE_URL = that Internal Database URL. Save.
7. Easier: on the Web Service, Environment, use Link Database if shown. Render fills DATABASE_URL.
8. Manual Deploy -> Deploy latest commit. Wait for green.
9. In logs you should see: Storage: PostgreSQL

If logs still say Storage: file ... then DATABASE_URL is missing or not visible to the web service. Link the database again and redeploy. Do not create a second web service.

4.3 Your three lasting links

If the service URL is <https://vedashri-travels.onrender.com> :

- Customer: <https://vedashri-travels.onrender.com/customer.html>
- Owner: <https://vedashri-travels.onrender.com/owner.html> PIN 2145
- App: <https://vedashri-travels.onrender.com/app.html>

Replace vedashri-travels with the name Render assigned. These URLs stay as long as that service is running.

4.4 Disk option (only if you skip Postgres)

Persistent disks are not on Render Free. Disks appears after you change Instance type from Free to Starter.

1. Settings -> Instance type -> Starter. Save.
2. Left sidebar Disks -> Add disk. Mount path /var/data. Size 1 GB.
3. Environment: DATA_DIR = /var/data
4. Redeploy.

Postgres (section 4.2) is the better free path. You do not need both.

4.5 After each code change

```
git add .  
git commit -m "Describe the change"  
git push
```

Render rebuilds the same service. The three URLs do not change.

5. Deploy on Railway (no Render upgrade)

1. Sign in at railway.app with GitHub.
2. New Project -> Deploy from GitHub repo -> vedashri-travels.
3. Railway detects Node. Start command: npm start. Build: npm install.
4. New -> Database -> PostgreSQL. Wait until it is running.
5. Open the web service -> Variables. Add DATABASE_URL from the Postgres plugin (Railway often injects it when you link the database). Do not set PORT.
6. Generate a public domain: Settings -> Networking -> Generate domain.
7. Share /customer.html, /owner.html and /app.html on that domain.

Optional: add a Volume at /var/data and set DATA_DIR=/var/data if you want file backups as well. Postgres alone is enough for bookings and fleet.

6. Deploy on a small VPS (Hetzner / DigitalOcean)

Use Ubuntu 22.04. Point a domain or use the server IP.

```
sudo apt update
sudo apt install -y nodejs npm nginx postgresql
sudo -u postgres createdb vedashri
sudo -u postgres createuser vedashri
# set a password in psql, then:
export DATABASE_URL=postgres://vedashri:PASSWORD@127.0.0.1:5432/vedashri
git clone https://github.com/YOUR_USER/vedashri-travels.git /var/www/vedashri
cd /var/www/vedashri
npm install
DATABASE_URL=$DATABASE_URL npx --yes pm2 start server.js --name vedashri
npx --yes pm2 startup && npx --yes pm2 save
```

Nginx (one public port, all three links):

```
server {
    listen 80;
    server_name travels.example.in;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

HTTPS: sudo apt install certbot python3-certbot-nginx && sudo certbot --nginx -d travels.example.in

7. What not to use

- Vercel / Netlify / GitHub Pages: no always-on Node process, no lasting store.json, uploads vanish.
- A second Render Web Service: you would get a new URL and empty data. Edit the existing service.
- MonkeyCode session preview: expires with the session. Not a production host.

8. Check that it worked

1. Open /customer.html. Book a test trip.
2. Open /owner.html, PIN 2145. Confirm the booking is listed. Change a car rate. Refresh the customer Fleet page. The new rate must show.
3. Render: Manual Deploy once. After it comes back, the booking and the new rate must still be there. If they vanished, DATABASE_URL is not set.
4. Logs should contain Storage: PostgreSQL

9. Owner first login

- PIN 2145. Change it under Company.
- Upload logo and GPay QR.
- Edit fleet rates. Customer site follows live.

10. Files in this project that matter for deploy

- package.json start: npm start Node 18+
- server.js listens on 0.0.0.0 and process.env.PORT
- lib/store.js PostgreSQL if DATABASE_URL, else JSON file
- render.yaml example Web Service + Postgres (optional Blueprint)
- public/customer.html public/owner.html public/app.html

Vedashri Travels - keep this PDF in docs/VEDASHRI_DEPLOY_RENDER.pdf